Computer Graphics

Primitive Pipeline

Dr. Hardik Jain
Assistant Professor
Department of Computer Science and Engineering
IIT Jodhpur

1

Primitives

The basic geometric building blocks—primarily points, lines, and triangles—that define the shapes which needs to be rendered

2

Primitive Pipeline

The sequence of processing stages that transforms geometric primitives from their 3D mathematical descriptions into 2D pixels on the screen

3

Primitive Pipeline Overview

- Vertex Specification
- Vertex Processing
- Primitive Assembly
- Clipping
- Culling
- Rasterization
- Fragment Processing
- Framebuffer Output

The primitive pipeline processes geometric data through several key stages, converting vertices and primitives from 3D world space into final rendered pixels in the framebuffer

4

Vertex Specification

Application Stage

- The application sets up an ordered list of vertices to send to the pipeline.
- These vertices define the boundaries of primitives (basic drawing shapes like triangles, lines, and points).

- Vertex Array Objects (VAOs):
  - Define what data each vertex has
  - Specify the layout of vertex attributes
- Vertex Buffer Objects (VBOs):
  - Store the actual vertex data itself
  - Hold arrays of attribute values
- Vertex Attributes:
  - Each vertex has a series of attributes (position, color, normal, texture coordinates, etc.)
  - Attribute data is arbitrary - you define what each vertex contains
  - There's no requirement that attributes must be position or normal

5

Vertex Processing

Vertex Shader

- Converts each incoming vertex into a single outgoing vertex
  - By independently applying Model, View and Projection matrices to the vertex
  - Can process different vertex attributes like colour, texture, normals
  - Operates in parallel on all vertices
- Doesn't know which other vertices form a triangle with it, i.e. No information about the connectivity
- **Modern GPUs**: This is the programmable part of the pipeline where user-written shaders control vertex transformation and can replace the standard processing

6

Primitive Assembly

Collect vertex data and compose it into a sequence of primitives

- **Establishes Topology**: Groups vertices into complete geometric primitives (points, lines, or triangles)
- Collect vertex data and compose it into a sequence of primitives
- Groups vertices according to the primitive type specified in the draw call
- Outputs an ordered sequence of simple primitives
- **Primitive Assembly is the essential bridge that converts an unstructured vertex stream into connected geometric primitives that clipping can operate on**

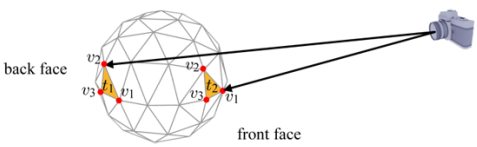
Clipping

Ensure all primitives lie within the viewing volume

- Primitives on the boundary between inside/outside the viewing volume are split into multiple primitives
- Primitives (parts of primitives) outside the viewing volume are clipped or removed entirely
- Ensures the entire primitive lies inside the volume
- Tests against 6 clipping planes (left, right, top, bottom, near, far)
- Vertex positions transformed from clip-space to window space

Back Face Culling

Discard triangles facing away from the viewer



- Triangle primitives can be culled based on their facing in window space
- Avoids rendering back-facing triangles
- For closed surfaces, back-facing triangles are naturally occluded by front-facing ones
- **Performance optimization** - skip rendering invisible geometry

9

Rasterization

Converts *continuous geometric primitives* into *discrete fragments*

- Fragment
  - A set of state used to compute the final data for a pixel (or sample if multisampling is enabled) in the output framebuffer
  - Essentially pixel-sized samples that contain position, color, depth, and texture information
  - Corresponds to a potential pixel on screen
- Rasterization bridges the gap between continuous geometric primitives and discrete pixel grid
- This is where shapes become pixels
- Evaluate which pixels are part of the primitive.
- Interpolate vertex attributes across fragments and queues for further processing.
- This is computationally quite expensive and is usually done by a dedicated hardware unit.
- For example, a triangle might cover 50 pixels, so rasterization creates 50 fragments to represent it.

10

Fragment Processing/Shader

Process each fragment to determine pixel color and depth

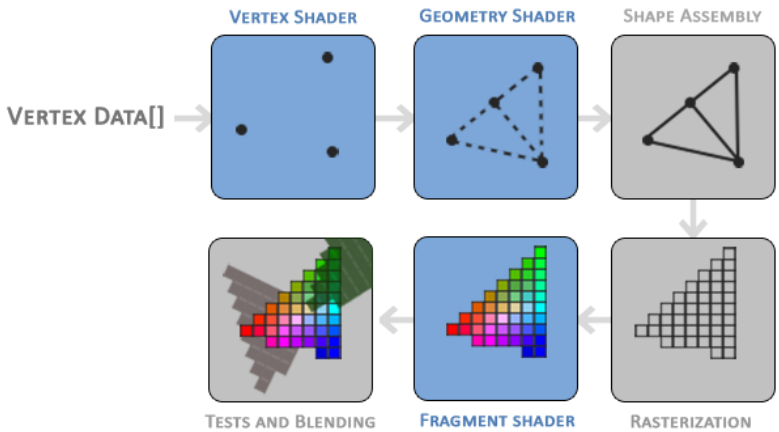
- Interpolated data from rasterization
- Calculates the final color for each fragment through texture mapping, lighting effects, and other per-pixel operations
- Performs depth testing, stencil testing, alpha blending, and other tests to determine which fragments actually update the framebuffer.
- For eg:
  - Add colors or textures (e.g., making a wall look like wood).
  - Apply lighting (e.g., adding shadows or highlights).
  - Create effects like transparency.

Framebuffer & Display

A collection of buffers in GPU memory that acts as a canvas for rendering

- Surviving fragments update pixel colors in the framebuffer, which is then sent to your display.
- It doesn't just store colours; it can hold various attachments:
  - **Color Buffer**: Stores the actual RGBA color of the pixels.
  - **Depth Buffer** (Z-Buffer): Stores depth values to determine which object is in front.
  - **Stencil Buffer**: Used to mask out certain areas, limiting where drawing can occur.

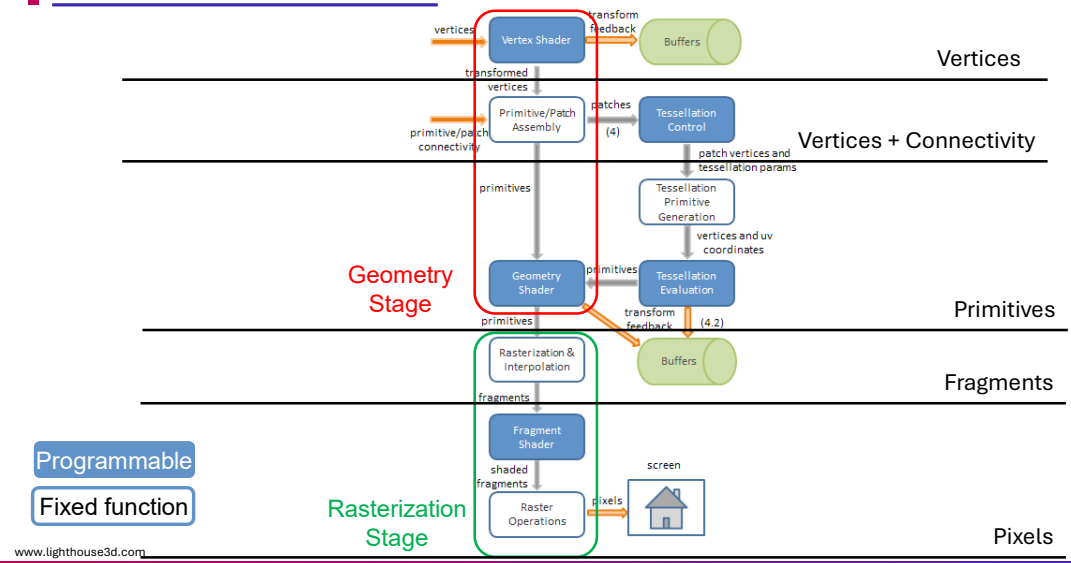
Summary



Programmable GPU's

- Graphics Processing Units are programmable today
- Novel shading and lighting can be performed by writing appropriate vetex and pixel shaders, beyond OpenGL
- GPUs used parallel processing with 2-4 vertex and 32-64 pixel processing units, all working in parallel.
- **Clever Idea**: Use the power for other processing: matrix multiplication, FFT, sorting, image processing, etc.
- GPGPU: General Processing on GPUs

OpenGL 3.2 and later



15

Thank You

16